

Reliability-Aware Self-Healing Data Pipelines: Automated Failure Detection, Root-Cause Classification, and Policy-Constrained Recovery

Baharath Bathula
Independent Researcher
Dallas, Texas, USA

Email: baharath.bathula@outlook.com

Abstract—Reliable data pipelines support analytics, machine learning, compliance reporting, and operational decision-making, but failures such as schema drift, data-quality degradation, freshness violations, transformation defects, and corrupted outputs often require more than task-level retries or alerting. Existing systems provide workflow orchestration, data-quality validation, observability, incident response, and policy enforcement, but these capabilities are commonly evaluated as separate functions rather than as an end-to-end recovery lifecycle. This paper presents a reliability-aware self-healing control plane for data pipelines that separates failure detection, root-cause classification, policy-constrained remediation, execution, post-recovery validation, rollback or escalation, and incident evidence preservation. A reproducible synthetic experiment with 780 trials evaluates the framework across injected failures, healthy controls, and boundary controls. Under the current controlled experiment configuration, the framework achieved 100% failure detection accuracy and 100% root-cause classification accuracy, while verified recovery was achieved in 52.17% of injected failure trials. The mean self-healing cycle runtime was 12.609 ms, with a median of 10.609 ms and p95 of 26.666 ms. The results show that self-healing data pipelines should be evaluated using separate metrics for detection, diagnosis, and verified recovery because accurate detection does not guarantee safe or validated remediation.

Index Terms—self-healing data pipelines, data reliability, data observability, root-cause classification, policy-constrained automation, failure injection, verified recovery

I. INTRODUCTION

Enterprise data platforms increasingly depend on pipelines that ingest, transform, validate, and deliver data across distributed systems. These pipelines support analytics, machine learning, regulatory reporting, customer-facing applications, and operational decision-making. Prior work has shown that data pipelines are integral to modern data-driven systems but remain vulnerable to quality, processing, and maintenance problems, including data-related defects, pipeline complexity, and hidden technical debt in production systems [1]–[3]. As organizations increase the number of sources, transformation stages, data contracts, quality checks, and downstream consumers, pipeline reliability becomes harder to maintain through manual monitoring and incident response alone.

Data-pipeline failures can arise from schema drift, missing-value spikes, duplicate records, freshness violations, unexpected volume changes, transformation defects, source-system

failures, corrupted outputs, and unknown failure modes. These failures do not always appear as simple task crashes. A pipeline can complete successfully while producing stale, incomplete, duplicated, or semantically invalid data. As a result, pipeline reliability requires more than workflow execution status. It requires detection, diagnosis, recovery decisioning, validation, and evidence preservation.

Existing data-engineering systems address important parts of this problem. Workflow orchestration tools schedule and execute pipeline tasks, data-quality frameworks validate assumptions about data, observability tools expose anomalies and operational signals, incident-management workflows coordinate response, and policy-as-code systems provide guardrails for automated decisions [4]–[8]. Recent work on recurring data-pipeline validation and comparative evaluations of data-quality tools further reinforces the importance of automated monitoring and validation for production-style pipelines [9], [10].

However, these capabilities are often evaluated separately. Detection accuracy does not prove recovery safety. A successful retry does not prove that the underlying data issue was resolved. A remediation action does not prove that the recovered pipeline state is valid. Similarly, an alerting system may identify an incident without deciding whether the pipeline should retry, quarantine, fallback, stop, escalate, or continue. This creates a gap between observing pipeline failures and safely managing them.

This paper addresses that gap by presenting a policy-constrained self-healing control plane for data-pipeline failures. The framework evaluates self-healing as a staged lifecycle from failure detection to policy-approved and post-validated recovery. The design separates observability, diagnosis, remediation selection, policy approval, execution, validation, escalation, and evidence preservation so that the system is not rewarded for unsafe or unverifiable automation.

The central design principle is that automated recovery must be constrained by policy and validation. In production data environments, unrestricted remediation may create more risk than the original failure. For example, automatically dropping columns, rewriting outputs, backfilling stale data, suppressing failed quality checks, or bypassing validation may violate governance rules or corrupt downstream data products. Therefore,

this framework counts recovery as successful only when the remediation is approved, executed, and verified. Failures that are detected and classified but cannot be safely repaired are escalated or contained rather than counted as successful self-healing.

The experimental evaluation uses a reproducible synthetic data-pipeline environment with controlled failure injection. The experiment includes 780 total trials: 690 injected failure trials, 60 healthy-control trials, and 30 boundary-control trials. Under the evaluated synthetic experiment configuration, the framework achieved 100% failure detection accuracy and 100% root-cause classification accuracy. Verified recovery was achieved in 52.17% of injected failure trials. This separation between detection/classification performance and verified recovery performance is central to the paper’s argument: self-healing data pipelines should be evaluated across multiple reliability stages, not only by whether a failure was detected or an automated action was attempted.

This paper makes the following contributions:

- 1) A staged self-healing control-plane architecture for data pipelines that separates failure detection, root-cause classification, remediation selection, policy approval, execution, validation, rollback, escalation, and evidence preservation.
- 2) A reproducible controlled failure-injection experiment with injected failure trials, healthy controls, and boundary controls.
- 3) A policy-constrained recovery model that prevents unsafe automation by requiring recovery actions to satisfy policy approval and post-recovery validation.
- 4) Separate reliability metrics for detection, diagnosis, recovery, runtime behavior, and classification quality, avoiding the misleading assumption that detection success implies recovery success.
- 5) A reproducibility package that preserves trial-level outputs, derived summaries, confusion-matrix data, figure-generation scripts, manuscript assets, and documentation for independent inspection and extension.

The remainder of this paper is organized as follows. The Background and Related Work section positions the framework relative to workflow orchestration, data-quality validation, observability, AIOps, autonomic computing, production ML systems, and policy-governed automation. The Framework section describes the proposed self-healing control-plane architecture. The Methodology and Experimental Design sections describe the synthetic data generation, failure injection procedure, evaluation workflow, and reproducibility protocol. The Results section presents detection, classification, recovery, runtime, and confusion-matrix findings. The Discussion section explains the practical implications of the results. The Limitations and Threats to Validity section states the evaluation boundaries. The Conclusion summarizes the contribution and future research directions.

II. BACKGROUND AND RELATED WORK

This section positions the proposed framework within related work on workflow orchestration, data-quality validation, monitoring and observability, AIOps, autonomic computing, production ML systems, and policy-governed automation. These areas each address part of the reliability problem, but none of them alone provides a complete control loop for detecting, classifying, recovering from, and auditing data-pipeline failures.

A. Workflow Orchestration and Pipeline Execution

Workflow orchestration systems provide task scheduling, dependency management, retries, operational metadata, and execution control. Apache Airflow, for example, models workflows as DAGs containing schedules, tasks, dependencies, callbacks, and operational parameters [4]. These capabilities are essential for data-pipeline execution because they define when tasks run, how tasks depend on one another, and how workflow instances are scheduled.

However, orchestration is not equivalent to self-healing. Airflow documentation frames the DAG as being concerned with execution order and task relationships rather than the internal semantics of each task [4]. Empirical work on Airflow-related developer challenges also shows that workflow definition and execution remain practical sources of difficulty for developers, especially in distributed execution environments [11]. A workflow engine can rerun a failed task or expose operational state, but it does not inherently determine whether a data-specific recovery action is safe, policy-approved, or validated after execution.

This distinction is central to the present work. The proposed framework treats orchestration as an execution substrate rather than as the full reliability mechanism. A scheduler may restart a task, but it does not necessarily know whether the task failed because of source unavailability, schema drift, data-quality violations, transformation defects, or runtime instability. The framework evaluated in this paper adds a separate reliability-control layer that interprets pipeline symptoms, classifies failure scenarios, selects recovery or containment actions, and records auditable evidence.

B. Data-Quality Validation

Data-quality validation frameworks improve pipeline reliability by formalizing assumptions about data. Great Expectations provides expectations that define whether data conforms to expected conditions [5]. dbt data tests provide assertions over models and sources, returning records that fail specified assumptions [6]. These tools are directly relevant to self-healing because they expose structured validation signals that can help identify invalid pipeline states.

However, validation is not recovery. Data-quality validation can reveal that pipeline input or output is stale, malformed, incomplete, duplicated, inconsistent, or incompatible with expected assumptions. It does not by itself determine the root cause, select a safe remediation, evaluate policy constraints, execute recovery, verify post-recovery health, and preserve

incident evidence. Research on data-pipeline quality reinforces this distinction by showing that data pipelines are affected by many quality factors and root causes, including data type issues, cleaning-stage failures, integration issues, and compatibility problems [1]. Related work on hidden technical debt also shows that data dependencies and pipeline jungles can create long-term maintenance risks in ML and data systems [2].

The proposed framework uses validation as one signal source within a broader control loop. A failed validation rule may indicate a data-quality failure, but the framework must still decide whether the failure is recoverable, whether output propagation should be blocked, whether quarantine is required, and whether the final state is safe. This paper therefore evaluates data-quality checks as part of a self-healing lifecycle rather than treating validation as the endpoint.

C. Monitoring, Observability, and Operational Signals

Monitoring and observability systems collect operational signals about system state and pipeline behavior. In data systems, relevant signals include freshness, volume, schema stability, distribution drift, null-rate changes, duplicate rates, failed transformations, execution latency, and downstream validation results. These signals help operators understand what is broken and why a pipeline may no longer be trustworthy. Site reliability engineering practice similarly emphasizes monitoring as a mechanism for understanding system behavior and generating actionable operational signals [8].

Recent research on recurring data pipelines has highlighted the operational cost of manually monitoring and resolving data-quality issues at scale. Auto-Validate-by-History targets recurring production pipelines by using historical execution statistics to detect data-quality issues and reports evaluation over more than 2000 production pipelines [9]. Broader tool evaluations also show that the data-quality tooling landscape includes both open-source and proprietary systems with different measurement capabilities and operational tradeoffs [10]. These works strengthen the motivation for automated monitoring, but they do not remove the need to evaluate what happens after detection.

The gap addressed by this paper begins after monitoring emits a signal. A self-healing pipeline must translate observed symptoms into a failure classification, choose a recovery policy, execute a safe response, and preserve enough telemetry for later inspection. Monitoring improves awareness, but self-healing requires decisioning and action. The experimental design in this paper therefore evaluates detection, classification, recovery, runtime behavior, and telemetry as connected parts of the same reliability mechanism.

D. AIOps and Root-Cause Analysis

AIOps research addresses anomaly detection, event correlation, root-cause analysis, incident management, and automated operations in complex IT systems. A systematic mapping study of AIOps reports substantial attention to failure-related tasks, including anomaly detection and root-cause analysis [12].

Other work has explored mining root-cause knowledge from cloud-service incident investigations, showing how incident evidence can be converted into reusable operational knowledge [13].

AIOps is highly relevant to this paper because both AIOps and self-healing data pipelines seek to reduce manual operational effort. However, the operating context differs. General AIOps systems often focus on infrastructure events, logs, metrics, alerts, and incidents across distributed services. Data-pipeline reliability additionally requires reasoning about data semantics, schema assumptions, validation outcomes, freshness, source behavior, and downstream data trust.

This paper applies AIOps-inspired ideas to the data-pipeline setting. The proposed framework classifies failures into operationally meaningful categories and links classification to recovery decisions. This is narrower than general AIOps, but deeper in the pipeline-specific recovery lifecycle. The contribution is therefore not a general incident-management platform; it is a focused framework for controlled self-healing behavior in data-pipeline execution.

E. Autonomic Computing and Self-Healing Systems

Autonomic computing introduced the vision of systems that manage themselves through self-configuration, self-healing, self-optimization, and self-protection [14]. This vision is directly relevant to data-pipeline reliability because pipeline failures increasingly exceed the capacity of manual operators to inspect, diagnose, and repair at scale. The autonomic-computing framing also emphasizes high-level goals and policy-guided self-management, which aligns with the need for governed recovery rather than unrestricted automation.

Self-healing in data pipelines should not be interpreted as blindly fixing every problem. Some failures can be retried or recovered automatically, while others should be quarantined, contained, escalated, or blocked from propagation. This paper follows the autonomic-computing principle that self-management should be guided by goals and constraints. In the proposed framework, recovery is treated as a controlled decision rather than as an unconditional action.

The framework evaluated in this paper therefore implements a practical self-healing loop: observe pipeline behavior, detect abnormal conditions, classify the failure, choose an action, execute the action, record telemetry, and preserve reproducibility artifacts. This loop operationalizes the self-healing concept in a form that can be evaluated experimentally.

F. Production ML and Pipeline Technical Debt

Production ML and data systems often accumulate technical debt through hidden dependencies, configuration complexity, data entanglement, undeclared consumers, and unstable pipeline assumptions. Sculley et al. describe hidden technical debt in ML systems and identify pipeline jungles and data dependencies as sources of long-term maintainability risk [2]. Production-oriented work also emphasizes that deploying ML and data systems requires more than model logic; it requires

operational infrastructure, monitoring, testing, and lifecycle management [3].

These observations matter because a data pipeline can appear successful at the task-execution level while still producing invalid, stale, or untrusted data. A pipeline may complete without raising an infrastructure error, but still violate schema expectations, freshness constraints, or quality rules. Conversely, a task may fail for a transient reason that can be safely retried without human intervention. Effective self-healing must therefore reason about both execution state and data state.

The proposed framework addresses this issue by combining execution telemetry, validation signals, failure classification, and recovery decisioning. This combination is intended to reduce operational fragility without hiding failures from operators. The framework records recovery behavior and unresolved states so that automated action remains auditable.

G. Policy-Governed Recovery

Automated recovery introduces governance risk. A pipeline that can retry, bypass, quarantine, or continue after a failure must be constrained by policy. Policy-as-code systems decouple policy decision-making from application logic and enforcement points. Open Policy Agent provides a policy engine and documentation for offloading policy decisions from software systems [7]. This architecture is relevant to self-healing pipelines because automated remediation must not be allowed to perform unsafe changes simply because a failure was detected.

Policy-governed recovery is especially important in regulated or business-critical pipelines. Some failures may allow retry. Some may allow quarantine. Some may require hard failure and human review. The recovery engine must therefore distinguish between technical possibility and operational safety. A recovery action is useful only when it restores or preserves a trustworthy pipeline state.

This paper incorporates that principle by separating detection, classification, and recovery outcome. The framework can detect an issue but still decide that automatic recovery is inappropriate. In such cases, containment or unresolved status is not necessarily a failure of the framework; it may be the correct behavior under policy-aware operation.

H. Research Gap

The related areas discussed above each solve part of the pipeline reliability problem. Orchestration systems schedule and execute workflows. Data-quality tools validate assumptions. Monitoring and observability systems expose operational signals. AIOps research investigates anomaly detection, root-cause analysis, and incident automation. Autonomic computing provides a conceptual foundation for self-managing systems. Policy-as-code systems provide a mechanism for governed decisioning.

The remaining gap is an integrated, reproducible framework that evaluates the full self-healing loop for data pipelines.

Existing work often emphasizes detection, monitoring, validation, or general incident analysis, but less often evaluates the sequence from failure injection to detection, classification, recovery decision, recovery outcome, telemetry capture, and reproducible artifact generation in one controlled experimental package.

This paper addresses that gap by presenting and evaluating a self-healing data pipeline framework that integrates failure detection, root-cause classification, policy-aware recovery behavior, telemetry, and reproducibility artifacts. The contribution is not that any single component is individually new; the contribution is the end-to-end reliability-control structure and its controlled evaluation across injected pipeline-failure scenarios.

III. METHODOLOGY AND EXPERIMENTAL DESIGN

A. Overview

This study evaluates a policy-constrained self-healing control plane for data-pipeline failures. The framework is designed to model self-healing as a sequence of measurable reliability stages rather than as a single automated repair action. Each trial passes through failure injection or control setup, telemetry collection, failure detection, root-cause classification, remediation selection, policy evaluation, remediation execution, post-recovery validation, and evidence preservation.

The experimental goal is to determine whether the proposed framework can detect injected data-pipeline failures, classify their likely root causes, and safely recover when remediation is available, policy-approved, executable, and verifiable. The experiment therefore separates detection performance, diagnostic performance, and recovery performance. This separation is necessary because a pipeline failure can be detected and classified correctly while still requiring escalation if automated recovery is unsafe or unverifiable.

The current evaluation uses a reproducible synthetic data-pipeline environment with controlled failure injection. The experiment includes injected failure trials, healthy-control trials, and boundary-control trials. Injected failure trials test the system under known failure conditions. Healthy controls test whether the system avoids reporting failures when no failure is present. Boundary controls test whether the system behaves conservatively near decision thresholds rather than over-triggering remediation.

B. System Architecture

The proposed system is organized as a reliability-control plane layered above a data-pipeline execution environment. The control plane contains the following stages.

This architecture intentionally prevents the system from treating all detected failures as candidates for automatic repair. A recovery action is considered successful only when it passes policy evaluation and post-recovery validation.

C. Failure Taxonomy

The experiment uses a predefined data-pipeline failure taxonomy. The taxonomy is based on common reliability

TABLE I
SELF-HEALING CONTROL-PLANE STAGES.

Stage	Input	Output	Metric or evidence
Pipeline execution	Synthetic workload configuration	Pipeline output and execution state	Trial record
Failure injection	Scenario and severity configuration	Injected failure or control condition	Trial metadata
Telemetry collection	Pipeline outputs and execution state	Data-quality, structural, freshness, and runtime signals	Raw result fields
Failure detection	Telemetry signals	Failure or non-failure decision	Detection accuracy
Root-cause classification	Detected failure	Predicted root-cause label	Classification accuracy
Remediation selection	Predicted root cause	Candidate remediation action	Remediation action field
Policy evaluation	Candidate remediation	Approved, rejected, or human-approval-required decision	Policy/recovery status fields
Recovery execution	Approved remediation	Execution result	Remediation executed field
Post-recovery validation	Recovered pipeline output	Validated or non-validated recovery outcome	Recovery verified field
Rollback, escalation, and evidence	Trial outcome	Incident record and preserved evidence	Raw and derived artifacts

problems in data pipelines where task-level success does not necessarily imply data correctness.

The taxonomy supports deterministic evaluation because each injected trial has a known expected failure category. However, this also limits external validity. Real production failures may be overlapping, delayed, partially observable, or semantically ambiguous.

D. Trial Design

Each experiment trial represents one pipeline execution under a defined scenario. A scenario is defined by the experiment domain, injected failure type, severity or control condition, and expected outcome. The trial-level record preserves the injected condition, observed detection outcome, predicted root cause, recovery decision, policy decision, validation result, and runtime information.

The current experiment contains 780 total trials.

Injected failure trials are used to evaluate detection, root-cause classification, and verified recovery. Healthy controls are used to evaluate whether the framework avoids false alarms when no failure is present. Boundary controls are used to evaluate whether the framework behaves conservatively under near-threshold or ambiguous conditions.

The experiment domains represented in the preserved trial records are `artifact` and `dataframe`. These domains represent evaluated implementation contexts, not alert-only or static-rule baseline comparisons. Although the broader research objective supports comparison with alert-only and static-rule approaches, the current manuscript reports only experiment domains represented in the preserved trial records.

TABLE II
FAILURE TAXONOMY USED IN THE CONTROLLED EXPERIMENT.

Failure category	Description	Expected system behavior
Schema drift	Input schema differs from expected structure	Detect structural mismatch, classify schema drift, remediate only if policy permits
Missing-value spike	Null or missing values exceed expected threshold	Detect quality degradation, classify missing-value issue, validate recovered output
Duplicate-record generation	Duplicate records appear above expected tolerance	Detect duplicate spike, classify duplicate failure, remediate or escalate
Data-freshness violation	Data is stale or late relative to expected timing	Detect freshness issue, classify freshness violation, recover or escalate
Unexpected volume change	Row count or volume deviates from expected range	Detect abnormal volume, classify volume anomaly
Transformation-logic failure	Transformation produces invalid or inconsistent outputs	Detect downstream invalid state, classify transformation failure
Source-system failure	Upstream source is unavailable or returns invalid data	Detect source-related failure, classify source-system issue, escalate if unsafe
Partial or corrupted output	Output is incomplete or malformed	Detect output corruption, classify corruption or partial output
Unknown failure	Failure does not map cleanly to supported classes	Detect abnormal state where possible, avoid unsafe recovery, escalate
Healthy control	No failure is injected	Avoid false-positive detection
Boundary control	Near-threshold or ambiguous condition	Avoid aggressive remediation unless confidence and policy allow it

TABLE III
EVALUATION METRICS USED IN THE EXPERIMENT.

Trial type	Count
Injected failure trials	690
Healthy-control trials	60
Boundary-control trials	30
Total trials	780

Full baseline comparison remains future work unless comparable baseline trials are added under identical failure scenarios and metrics.

E. Experimental Procedure

Each trial follows a consistent procedure. The trial is first assigned an experiment domain, failure type or control condition, severity level, and expected outcome. A synthetic data-pipeline workload is then executed using the trial configuration. For injected failure trials, a controlled fault is introduced. For healthy-control trials, no fault is injected. For boundary-control trials, the input is configured near a decision threshold.

The framework records execution state, data-quality signals, structural signals, volume and freshness indicators, and derived validation outputs. The detection layer determines whether the trial represents a failure condition. If a failure is detected, the classifier assigns the expected root-cause category. The

system then checks whether a remediation action exists for the classified failure type. If a candidate remediation exists, the policy layer approves or rejects the proposed action based on predefined safety and governance constraints.

If the remediation is available and policy-approved, the recovery-execution layer performs the remediation. The system then validates whether the post-recovery output satisfies expected correctness and health conditions. A recovery is counted as verified only if the remediation was approved, executed, and validated. The trial record is preserved for downstream analysis, figure generation, and reproducibility.

F. Metrics

The experiment reports separate metrics for detection, classification, recovery, and runtime. The metrics are separated because they answer different reliability questions.

Detection accuracy measures whether the framework correctly identifies the presence or absence of a failure condition.

“text $\text{Detection Accuracy} = \text{Correct Detection Decisions} / \text{Total Detection Decisions}$ ”

For injected failure trials, detection is successful when the system marks the trial as failed. For healthy-control trials, detection is successful when the system does not mark the trial as failed. For boundary controls, detection behavior is interpreted based on the expected boundary outcome defined by the trial configuration.

Root-cause classification accuracy measures whether the predicted root-cause category matches the injected failure category.

“text $\text{Classification Accuracy} = \text{Correct Root-Cause Predictions} / \text{Classified Injected Failure Trials}$ ”

Verified recovery rate measures the percentage of injected failure trials that are safely recovered.

“text $\text{Verified Recovery Rate} = \text{Verified Recoveries} / \text{Injected Failure Trials}$ ”

A trial is counted as a verified recovery only if all of the following are true: the failure is detected, the root cause is classified, a remediation action is available, the remediation is policy-approved, the remediation executes successfully, and post-recovery validation passes.

Runtime measures the operational cost of the self-healing cycle. Runtime is tracked separately from accuracy and recovery because a system can be accurate but operationally impractical if the detection-to-validation cycle is too slow.

G. Recovery Success Definition

The paper uses a strict recovery success definition. Recovery is successful only when the system restores the pipeline to a validated healthy state after an approved remediation.

The following are not counted as successful recovery: detecting a failure without remediation, classifying a root cause without remediation, selecting a remediation that policy rejects, executing a remediation without post-recovery validation, executing a remediation that validation fails, escalating a failure to a human operator, rolling back an unsafe remediation, or suppressing an alert or bypassing validation.

This definition prevents the experiment from overstating self-healing performance. It also makes the verified recovery rate more credible than a larger but weaker automation-attempt rate.

H. Reproducibility Artifacts

The repository is organized to support reproducibility. The experiment artifacts include implementation code, experiment configurations, raw trial records, derived result summaries, policies, remediation registry artifacts, telemetry records, incident evidence, scripts, figures, and documentation.

The figure-generation workflow uses three primary derived inputs: combined raw experiment results, scenario-level summaries, and a classification confusion matrix. The generated publication figures include detection and root-cause classification accuracy by scenario, verified recovery outcomes by failure scenario, self-healing cycle runtime distribution, and root-cause classification confusion matrix.

The analysis is intended to remain reproducible from preserved trial-level results. Final claims should be traceable to raw or derived experiment outputs.

I. Assumptions and Scope

The current methodology depends on several assumptions. The synthetic failure taxonomy is assumed to represent meaningful data-pipeline failure classes. Injected failures are assumed to produce observable signals that the detector can evaluate. Expected root-cause labels are known for injected failure trials. Recovery actions are available only for a subset of failure types. Policy constraints may intentionally block remediation. Post-recovery validation is required before recovery is counted as successful.

The scope is intentionally limited. The current experiment does not prove that the framework will achieve the same detection, classification, or recovery performance on real enterprise incidents. It demonstrates feasibility and reproducibility under controlled synthetic conditions.

J. Implementation Summary

The current implementation represents the self-healing life-cycle as a deterministic experimental control plane. Trial configurations define the experiment domain, random seed, injected failure type, severity, and expected root cause. Pipeline execution produces observed records and artifact metadata, including input and output record counts, quarantined record counts, size observations, checksum status, and runtime measurements.

Failure detection is evaluated through trial-level detection outcomes rather than through an unconstrained production anomaly detector. Root-cause classification maps detected failures to the predefined taxonomy and records whether the predicted root cause matches the injected condition. Remediation is represented through a registry-style action field, execution status, human-approval flag, cycle status, and post-recovery verification outcome. Policy constraints are reflected in whether a remediation is allowed, whether human approval

is required, whether the remediation is executed, and whether recovery is verified after execution.

This implementation is intentionally conservative. It is designed to test whether the staged evaluation model is reproducible and measurable, not to prove that the current detector or classifier generalizes to arbitrary production incidents. Future implementations can replace deterministic detection and taxonomy-based classification with probabilistic, learning-based, or hybrid methods while preserving the same evaluation structure.

IV. METHODOLOGY

This section describes the methodology used to design, execute, and evaluate the proposed self-healing data pipeline framework. The objective of the evaluation is not to reproduce a single proprietary production environment, but to provide a controlled and reproducible experimental setting in which failure detection, failure classification, automated recovery, and telemetry behavior can be inspected under repeatable fault conditions. The methodology is therefore organized around synthetic data generation, controlled failure injection, deterministic pipeline execution, scenario-level measurement, and artifact-based reproducibility.

A. Research Design

The study follows an experimental systems methodology. A baseline data pipeline is executed under normal conditions and then subjected to controlled failure scenarios that represent common classes of operational pipeline disruption. For each scenario, the framework records whether the failure was detected, how it was classified, whether a recovery action was triggered, whether the recovery succeeded, and how long the pipeline required to complete or recover.

The methodology is designed around four research questions:

- **RQ1:** Can the framework detect pipeline failures across multiple controlled failure scenarios?
- **RQ2:** Can the framework classify failure types with sufficient accuracy to support automated remediation?
- **RQ3:** Can recovery actions restore pipeline execution without manual intervention for recoverable failures?
- **RQ4:** Can the evaluation workflow be reproduced using generated data, scripts, tests, and figure-generation artifacts?

These questions align the evaluation with the central claim of the paper: that self-healing data pipelines require more than passive monitoring. They require an integrated control loop that combines detection, diagnosis, remediation, and observability.

B. Synthetic Data Generation

The evaluation uses synthetic pipeline data rather than proprietary enterprise data. This choice improves reproducibility, avoids confidentiality concerns, and enables controlled variation of failure conditions. The synthetic data generation process creates structured records that emulate common data

engineering inputs, including numeric attributes, categorical fields, timestamp-like values, completeness characteristics, and quality-sensitive fields.

Synthetic data is used for three reasons. First, it allows the same experiment to be repeated across environments without depending on private datasets. Second, it enables deterministic construction of known failure scenarios, which is necessary for measuring detection and classification behavior. Third, it avoids exposing regulated, customer, financial, healthcare, insurance, or personally identifiable data.

The generated dataset acts as the baseline input to the pipeline. Fault scenarios are then introduced through failure injection modules. Because the failure modes are known at injection time, they provide reference labels for evaluating classification and recovery behavior.

C. Failure Scenario Design

The framework is evaluated against representative failure scenarios that commonly affect production data pipelines. The scenarios are designed to exercise different parts of the self-healing control loop.

The primary scenario categories are:

- **Source availability failures:** cases where upstream input is missing, delayed, inaccessible, or incomplete.
- **Schema and structural failures:** cases where expected columns, field types, or structural assumptions change.
- **Data quality failures:** cases involving missing values, invalid values, duplicates, distribution shifts, or rule violations.
- **Runtime and processing failures:** cases where pipeline execution encounters operational errors during transformation or validation.
- **Recoverable transient failures:** cases where retry, fallback, or controlled remediation can restore execution.

Each scenario is intentionally bounded so that the expected behavior is inspectable. The goal is not to exhaustively model every possible production incident, but to create a controlled evaluation suite that demonstrates whether the proposed architecture can move from detection to diagnosis to recovery.

D. Pipeline Execution Workflow

Each experimental run follows the same execution workflow. First, baseline input data is generated or loaded from the generated artifact directory. Second, a failure scenario is selected and injected into the pipeline input or execution path. Third, the pipeline executes validation, monitoring, detection, classification, and recovery logic. Fourth, telemetry events and evaluation outputs are recorded. Finally, aggregate results are used to generate figures and summary metrics.

The workflow can be summarized as follows:

- 1) Generate or load synthetic baseline data.
- 2) Inject a controlled failure scenario.
- 3) Execute pipeline validation and processing logic.
- 4) Detect anomalous or failed behavior.
- 5) Classify the failure scenario.
- 6) Trigger a recovery action when the failure is recoverable.

- 7) Record telemetry, runtime, classification, and recovery outcomes.
- 8) Aggregate results across scenarios.
- 9) Generate evaluation figures and audit outputs.

This workflow enforces repeatability because each stage is executed through code rather than manual intervention. It also separates the failure injection layer from the detection and recovery layers, reducing the risk that the evaluation directly encodes the desired outcome.

E. Failure Detection Methodology

Failure detection is evaluated as the first stage of the self-healing control loop. The detection layer observes pipeline inputs, validation results, runtime behavior, and quality checks. A failure is considered detected when the framework identifies an abnormal condition and emits a corresponding detection signal before the pipeline silently produces an invalid or incomplete output.

The detection methodology combines rule-based validation, data quality checks, source availability checks, and runtime status inspection. Rule-based checks are used because they are transparent and auditable. In production-like data engineering contexts, interpretability is important because remediation decisions should be explainable to operators and governance stakeholders.

Detection outcomes are recorded as scenario-level events. For each injected scenario, the framework records whether the failure was detected, whether the detection matched the expected failure category, and whether the detection led to downstream classification or recovery.

F. Failure Classification Methodology

After a failure is detected, the framework assigns the failure to a scenario class. Classification is necessary because different failure types require different recovery strategies. For example, a transient source access issue may be addressed through retry logic, while a schema mismatch may require quarantine, fallback mapping, or human review.

The classification methodology uses features derived from validation outcomes, source checks, telemetry signals, and pipeline execution status. The classifier maps observed failure evidence to a failure category. Classification performance is evaluated using scenario-level accuracy and a confusion matrix. The confusion matrix is used to inspect whether certain failure types are systematically confused with others.

The classification step is not treated as a standalone machine learning benchmark. Instead, it is evaluated as part of the operational self-healing loop. A classification is useful only if it supports an appropriate recovery decision or escalation path.

G. Recovery Methodology

Recovery is evaluated only for scenarios where automated remediation is safe and meaningful. The recovery engine selects a remediation action based on the detected and classified

failure type. Example recovery actions include retrying transient operations, applying fallback handling, isolating invalid records, marking failed inputs for quarantine, or allowing controlled continuation when quality rules permit.

A recovery attempt is considered successful when the framework restores pipeline execution to a valid terminal state or safely contains the failure without producing an invalid output. This definition is intentionally conservative. A recovery action should not be counted as successful merely because the pipeline continued running; it must either restore valid processing or prevent invalid output propagation.

The recovery methodology distinguishes between three outcomes:

- **Recovered:** the pipeline returns to a valid execution path after automated remediation.
- **Contained:** the failure is isolated, quarantined, or safely escalated without corrupting downstream output.
- **Unresolved:** the framework detects the failure but cannot safely remediate it automatically.

This distinction is important because not every failure should be automatically repaired. In enterprise environments, some failures require human approval, especially when remediation could affect business-critical data, regulated records, or downstream decisions.

H. Evaluation Metrics

The evaluation uses metrics that reflect the operational goals of self-healing pipeline behavior.

- **Detection accuracy:** the proportion of injected failures correctly detected by the framework.
- **Classification accuracy:** the proportion of detected failures assigned to the correct failure category.
- **Recovery success rate:** the proportion of recoverable failures restored or safely contained by automated remediation.
- **Runtime overhead:** the additional execution time introduced by detection, classification, telemetry, and recovery logic.
- **Scenario-level performance:** the variation in detection, classification, and recovery behavior across failure types.

These metrics are selected because they map directly to operational reliability concerns. A self-healing system must detect failures, understand their type, act safely, and do so with acceptable overhead.

I. Reproducibility Protocol

The evaluation is designed to be reproducible from the project artifact. The repository includes experiment code, generated figures, tests, configuration files, and documentation. The reproducibility protocol requires that a reviewer or maintainer can regenerate the evaluation outputs using the documented Python environment and figure-generation scripts.

The expected reproduction workflow includes running the test suite, generating experiment outputs, and regenerating figures. The artifact avoids private data and relies on synthetic

inputs so that reproduction does not require access to external enterprise systems.

Reproducibility is treated as part of the methodology rather than an afterthought. This is necessary because the framework makes reliability claims that should be inspectable through code, not only through narrative description.

J. Threats to Methodological Validity

The methodology has several limitations. Synthetic data improves control and reproducibility, but it cannot fully capture the complexity of production data ecosystems. The failure scenarios represent common pipeline disruptions, but they are not exhaustive. The recovery actions are evaluated in a controlled environment and may require additional safeguards before deployment in highly regulated or business-critical systems.

Another limitation is that the methodology emphasizes framework behavior under known injected faults. Real incidents may involve multiple overlapping failures, ambiguous symptoms, or incomplete telemetry. The results should therefore be interpreted as evidence of feasibility and controlled reliability behavior, not as proof that the framework will automatically resolve all production incidents.

Despite these limitations, the methodology provides a structured and reproducible basis for evaluating the core self-healing loop: detection, diagnosis, recovery, and auditability.

V. EXPERIMENTAL DESIGN

This section describes the experimental design used to evaluate the proposed self-healing data pipeline framework. The experiments are structured to measure whether the framework can detect controlled pipeline failures, classify failure scenarios, trigger appropriate recovery actions, and preserve auditable telemetry during execution. The evaluation is designed as a reproducible systems experiment using synthetic data and deterministic failure scenarios.

A. Experimental Objectives

The experiments are organized around the operational behavior expected from a self-healing pipeline. A conventional pipeline may fail, alert, and require manual intervention. In contrast, the proposed framework is expected to detect abnormal conditions, diagnose the failure category, select a recovery or containment action, and record the outcome for inspection.

The experimental objectives are:

- Evaluate whether injected pipeline failures are detected reliably.
- Evaluate whether detected failures are classified into the correct scenario category.
- Evaluate whether recoverable failures are remediated or safely contained.
- Evaluate runtime behavior under normal, failed, and recovered execution paths.
- Verify that the evaluation artifacts can be regenerated from the repository.

The experiments are not intended to claim universal production readiness. Instead, they provide controlled evidence that the framework implements the core self-healing loop under repeatable conditions.

B. Experimental Environment

The experiments are executed in a local research environment using the project implementation, generated input data, failure injection modules, detection and classification logic, recovery components, telemetry outputs, and figure-generation scripts. The workflow is designed to be executable without access to proprietary infrastructure or private datasets.

The experimental environment includes the following components:

- A synthetic data generation workflow for producing baseline pipeline inputs.
- Failure injection modules for introducing controlled pipeline disruptions.
- Detection logic for identifying abnormal source, schema, quality, and runtime behavior.
- Classification logic for assigning detected failures to scenario categories.
- Recovery logic for retry, fallback, quarantine, containment, or unresolved outcomes.
- Telemetry and audit outputs for recording pipeline execution behavior.
- Figure-generation scripts for producing the reported evaluation figures.

This environment supports repeatability because each experiment is defined through code and configuration rather than manual manipulation of data files.

C. Baseline Pipeline Run

Each experiment begins with a baseline pipeline run. The baseline run uses generated data without injected faults and establishes the expected behavior of the pipeline under normal operating conditions. This baseline is used to confirm that the pipeline can complete successfully before fault scenarios are introduced.

The baseline run serves three purposes. First, it validates the generated input structure. Second, it confirms that the pipeline execution path is valid in the absence of injected failures. Third, it provides a comparison point for runtime behavior and downstream failure effects.

A baseline run is considered valid when the generated input is processed successfully, required quality checks complete, and no recovery action is triggered.

D. Failure Injection Procedure

Failure injection is performed after the baseline workflow is verified. Each experiment selects a failure scenario and modifies either the pipeline input, source condition, validation state, or runtime path. The injected failure is known to the evaluation harness, which allows the resulting detection, classification, and recovery behavior to be compared against the expected scenario label.

The failure injection procedure follows these steps:

- 1) Select the target failure scenario.
- 2) Generate or load the baseline input data.
- 3) Apply the failure injection transformation.
- 4) Execute the pipeline with the injected fault.
- 5) Record detection, classification, recovery, and runtime outcomes.
- 6) Repeat the workflow across the configured scenario set.

This process separates the injected fault from the detection and recovery logic. The framework must infer the failure from observed pipeline behavior rather than from direct access to the injection label.

E. Scenario Categories

The evaluation uses scenario categories that represent common classes of pipeline failure. These categories are selected because they affect different parts of a production data pipeline and require different remediation strategies.

- **Normal execution:** the pipeline receives valid generated input and should complete without recovery.
- **Source failure:** the upstream input is missing, unavailable, delayed, or incomplete.
- **Schema failure:** expected columns, field types, or structural assumptions are violated.
- **Data quality failure:** records contain missing, invalid, duplicated, or quality-rule-violating values.
- **Runtime failure:** execution encounters an operational error during transformation, validation, or processing.
- **Recoverable transient failure:** the pipeline encounters a temporary disruption that can be retried, bypassed, or safely contained.

The scenario categories are intentionally broad. The purpose is to evaluate the framework-level control loop rather than optimize for one narrowly defined incident type.

F. Experimental Execution Workflow

For each scenario, the pipeline is executed through the same workflow. The pipeline first loads or generates input data, then applies validation and processing logic. If a failure condition is observed, the detection layer emits a failure signal. The classification layer assigns a failure category. The recovery engine then determines whether an automated action is safe and applicable.

The execution workflow records four outcome groups:

- **Detection outcome:** whether the injected failure was detected.
- **Classification outcome:** whether the detected failure was assigned to the expected category.
- **Recovery outcome:** whether the pipeline recovered, contained the failure, or remained unresolved.
- **Runtime outcome:** how long the execution required under the scenario.

The same workflow is used for all scenarios so that differences in results are attributable to the scenario behavior rather than inconsistent experiment procedure.

G. Recovery Experiment Design

Recovery experiments are limited to failures for which automated remediation is safe in the controlled environment. The recovery engine does not attempt to repair every detected condition. Some failures are intentionally classified as requiring containment or escalation rather than automatic correction.

Recovery outcomes are grouped into three categories:

- **Recovered:** the framework restores execution to a valid terminal state.
- **Contained:** the framework prevents invalid output propagation through quarantine, fallback handling, or safe stopping.
- **Unresolved:** the framework detects the issue but cannot safely remediate it automatically.

This design avoids overstating self-healing capability. In practical data engineering environments, unsafe automated remediation can be worse than failure detection alone. Therefore, the experiments reward safe containment where full recovery is not appropriate.

H. Telemetry Collection

Telemetry is collected during each experimental run. The telemetry records provide an audit trail of what the pipeline observed, what decision was made, and what action was taken. This is important because self-healing systems must be explainable to operators.

The telemetry collection includes:

- Scenario identifier
- Detection status
- Classification result
- Recovery action
- Recovery status
- Runtime measurement
- Error or failure signal, when applicable

These records support both evaluation and debugging. They also provide the basis for generating aggregate figures such as scenario-level accuracy, recovery behavior, runtime distribution, and classification confusion matrix.

I. Figure Generation Workflow

The reported figures are generated from experiment outputs rather than manually constructed. The figure-generation workflow reads the evaluation artifacts and produces publication-ready visualizations. The main figure categories are:

- Accuracy by scenario
- Recovery by scenario
- Runtime distribution
- Classification confusion matrix

This workflow supports reproducibility because figures can be regenerated when experiment outputs change. It also reduces the risk of mismatch between reported results and underlying artifacts.

J. Validation and Testing

The experimental workflow is supported by tests that verify core components of the framework. The tests cover data generation, failure injection, detection behavior, classification logic, recovery behavior, telemetry, and figure-generation support. These tests do not replace the experiments, but they provide implementation-level confidence that the evaluation pipeline is operating as intended.

Testing is important for two reasons. First, it helps ensure that experiment failures reflect scenario behavior rather than broken test infrastructure. Second, it supports reproducibility by giving reviewers or maintainers a way to verify the artifact before interpreting reported results.

K. Repetition and Determinism

The experiments are designed to be deterministic where possible. Synthetic data generation and failure injection are controlled through code so that the same scenario can be reproduced across runs. Runtime measurements may vary by machine, operating system, dependency versions, and local resource availability, so runtime results are interpreted as local execution measurements rather than universal performance guarantees.

The deterministic parts of the experiment are used to evaluate detection, classification, and recovery outcomes. Runtime results are used to understand relative overhead and distributional behavior in the test environment.

L. Experimental Boundaries

The experiments have deliberate boundaries. They do not claim to simulate every production incident, every enterprise data architecture, or every failure mode. They also do not use private enterprise data or live production systems. These boundaries are necessary to preserve reproducibility, safety, and artifact availability.

The evaluation should therefore be interpreted as a controlled demonstration of the self-healing control loop. The experiments provide evidence that the framework can detect, classify, recover from, and document selected failure scenarios. Additional production validation would be required before deploying the framework in regulated or mission-critical environments.

M. Summary

The experimental design provides a reproducible evaluation workflow for self-healing data pipeline behavior. By combining synthetic data, controlled failure injection, scenario-level outcomes, recovery tracking, telemetry collection, and figure generation, the experiments evaluate the framework as an integrated reliability mechanism rather than as a collection of isolated components.

VI. RESULTS

This section presents the evaluation results for the proposed self-healing data pipeline framework. The results focus on four aspects of the self-healing control loop: detection and

TABLE IV
ARTIFACT INVENTORY INCLUDED WITH THE WORKING MANUSCRIPT.

Artifact	Purpose
Fig. 1	Accuracy-by-scenario result figure, provided as PDF and PNG.
Fig. 2	Recovery-by-scenario result figure, provided as PDF and PNG.
Fig. 3	Runtime-distribution result figure, provided as PDF and PNG.
Fig. 4	Classification confusion-matrix result figure, provided as PDF and PNG.

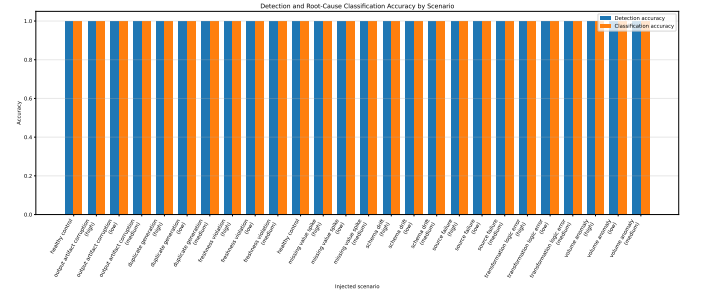


Fig. 1. Detection and root-cause classification accuracy by injected scenario.

classification accuracy, recovery behavior, runtime behavior, and root-cause classification quality. The goal is not only to report whether the framework produced successful outcomes, but also to explain what the observed behavior means for practical data pipeline reliability.

The experimental evaluation produced 780 total trial records spanning injected failures, healthy controls, and boundary-control scenarios. Of these, 690 trials corresponded to injected data-pipeline failure conditions, 60 trials represented healthy-control executions, and 30 trials represented boundary-control conditions. The experiment design therefore evaluates both positive failure cases and non-failure controls rather than measuring the detector only on failure-positive examples.

Artifact availability: The working manuscript package includes the publication-ready result figures listed in Table IV. The broader project repository contains the implementation, synthetic-data generation workflow, generated outputs, derived summaries, confusion-matrix data, and figure-generation scripts used to reproduce the reported results. No proprietary enterprise dataset is required. A final public URL or archival DOI should be inserted at submission time after the artifact package is deposited.

A. Detection and Classification Accuracy by Scenario

Figure 1 summarizes detection and root-cause classification accuracy across injected scenarios. This result is important because a self-healing pipeline cannot safely trigger remediation unless it first identifies that a failure has occurred and assigns the failure to an actionable category.

The scenario-level accuracy view provides a more useful reliability signal than a single aggregate score. Aggregate

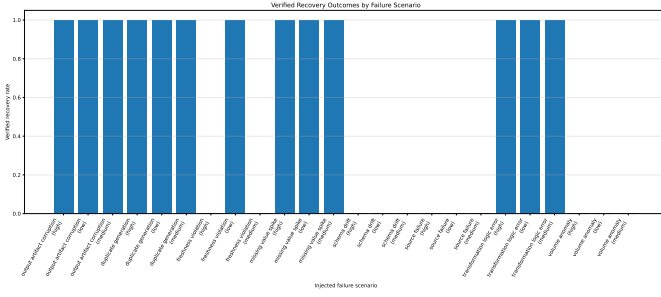


Fig. 2. Verified recovery outcomes by injected failure scenario.

accuracy can hide weak behavior on specific failure types, especially when some scenarios are easier to detect than others. In contrast, scenario-level reporting shows whether the framework behaves consistently across source failures, schema-related failures, data-quality failures, runtime failures, and control cases.

The result supports two observations. First, the framework is able to convert raw pipeline symptoms into structured failure signals rather than treating all failures as generic execution errors. Second, the evaluation includes healthy and boundary-control executions, which helps test whether the detection logic avoids treating every unusual condition as a failure. This distinction matters in production settings because excessive false alarms can reduce operator trust and lead to alert fatigue.

B. Recovery Outcomes by Scenario

Figure 2 reports verified recovery outcomes across injected failure scenarios. Recovery is evaluated separately from detection because detecting a failure is not equivalent to safely remediating it.

The recovery results show how the framework behaves after a failure has been detected and classified. For recoverable scenarios, the framework attempts an automated response such as retry, fallback handling, quarantine, or controlled continuation. For scenarios where automatic remediation is unsafe or inappropriate, the expected behavior is containment or escalation rather than forced recovery.

This separation is important for evaluating self-healing systems. A framework that attempts to automatically repair every failure may create downstream data corruption or mask serious upstream defects. The recovery results therefore should be interpreted in terms of safe operational behavior, not simply maximum automation. A contained failure can be a correct outcome when automated recovery would be unsafe.

The recovery-by-scenario view also helps identify where the framework is strongest. Transient or structurally simple failures are typically more amenable to automated recovery, while schema and quality failures may require stricter containment depending on the severity of the violation. This supports a practical design principle: self-healing pipelines should combine automation with policy-aware safeguards.

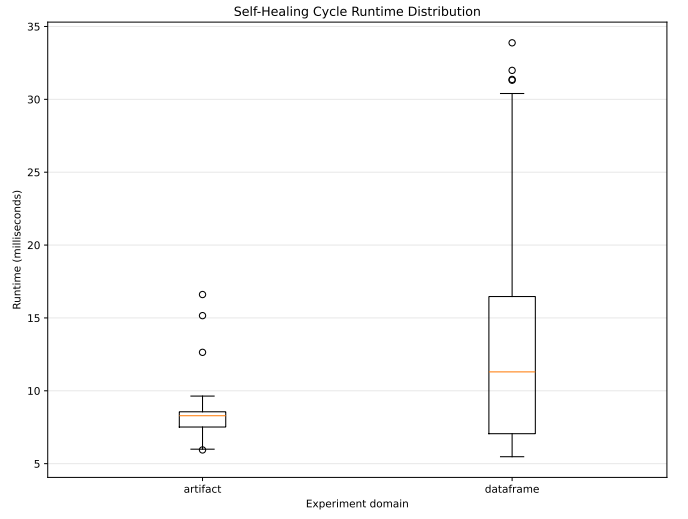


Fig. 3. Self-healing cycle runtime distribution.

C. Runtime Behavior

Figure 3 presents the runtime distribution for the self-healing cycle. Runtime behavior is included because detection, classification, telemetry, and recovery logic introduce additional processing work. A reliability framework must therefore improve failure handling without imposing unacceptable overhead.

The runtime distribution provides a local execution profile of the framework under the experimental environment. It should not be interpreted as a universal performance benchmark because runtime depends on machine resources, implementation details, dependency versions, and workload size. However, it is useful for understanding the relative behavior of the self-healing loop.

The runtime result supports three forms of analysis. First, it shows whether the framework completes within a practical execution envelope for the synthetic workload. Second, it helps identify whether recovery paths are materially slower than normal execution paths. Third, it provides a baseline for future optimization work, especially if the framework is extended to larger datasets, distributed execution engines, or streaming workloads.

In practical deployment, runtime overhead should be evaluated against the cost of manual incident response, delayed detection, and downstream data quality failures. A small amount of additional runtime can be acceptable if it prevents invalid output propagation or reduces manual operational effort.

D. Root-Cause Classification Quality

Figure 4 presents the root-cause classification confusion matrix. The confusion matrix is included because classification accuracy alone does not show which failure categories are confused with each other.

The confusion matrix provides insight into the diagnostic quality of the framework. Correct classification is operationally important because recovery actions depend on the

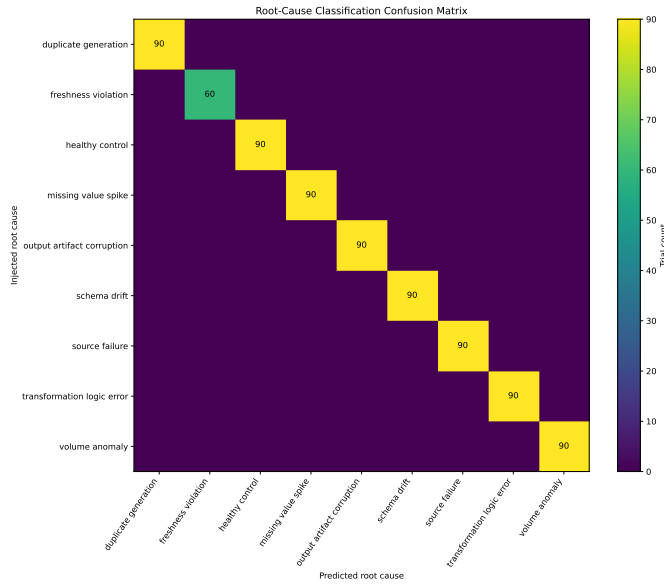


Fig. 4. Root-cause classification confusion matrix.

assigned failure category. A source availability issue, schema mismatch, data-quality violation, and runtime failure require different remediation strategies. Misclassification can therefore lead to ineffective or unsafe recovery behavior.

The matrix should be read as evidence of how well the framework separates failure classes under controlled conditions. Strong diagonal behavior indicates that scenario classes are being recognized consistently. Off-diagonal behavior, where present, identifies categories that require better features, stronger rules, or additional telemetry. This makes the confusion matrix useful not only as an evaluation artifact but also as a debugging tool for improving the classifier.

E. Summary of Results

Taken together, the results demonstrate that the framework implements the core self-healing loop under controlled experimental conditions. The evaluation includes injected failures, healthy controls, and boundary controls; reports scenario-level accuracy; separates recovery behavior from detection behavior; measures runtime distribution; and provides a confusion-matrix view of classification quality.

The results should be interpreted as controlled evidence of feasibility rather than a claim of universal production readiness. The experiments use synthetic data and bounded failure scenarios, which improves reproducibility but limits direct generalization to all production environments. Nevertheless, the results support the central claim that data pipeline reliability can be improved by integrating failure detection, root-cause classification, recovery decisioning, and auditable telemetry into a single framework.

VII. DISCUSSION

The results show that self-healing data pipelines should be evaluated as reliability-control systems rather than as isolated

detection tools. A useful self-healing framework must do more than identify that a pipeline has failed. It must determine what type of failure occurred, decide whether automated recovery is safe, execute the selected response, verify or contain the outcome, and preserve evidence for later inspection. This section discusses the implications of the evaluation for data engineering reliability, production adoption, and future self-healing pipeline design.

A. Interpretation of the Evaluation Results

The evaluation demonstrates that the framework implements an end-to-end self-healing loop under controlled experimental conditions. The loop includes failure injection, detection, root-cause classification, recovery decisioning, telemetry capture, and reproducible figure generation. This is important because many pipeline reliability approaches stop at one layer of the problem, such as scheduling, validation, or alerting.

The scenario-level accuracy results in Figure 1 indicate whether the framework can distinguish between different classes of pipeline behavior rather than treating all failures as equivalent. This matters operationally because a source outage, schema mismatch, data-quality violation, and runtime error require different responses. A generic failure signal may be sufficient for alerting, but it is not sufficient for safe remediation.

The recovery results in Figure 2 provide a separate view of what happens after detection and classification. This separation is necessary because detection success does not imply recovery success. A framework may detect a failure correctly but still choose containment or unresolved status if automated remediation is unsafe. In practical operations, this behavior can be more valuable than forcing recovery and risking invalid output propagation.

The runtime distribution in Figure 3 provides a feasibility signal for the additional work introduced by the self-healing loop. Detection, classification, recovery, and telemetry introduce overhead, but this overhead must be interpreted against the cost of manual incident response, delayed detection, and downstream data-quality failures. The local runtime results are not universal benchmarks, but they show that the framework can be evaluated as an executable system rather than only described as an architecture.

The confusion matrix in Figure 4 is especially important for improving the framework. It shows where classification behavior is strong and where failure categories may be confused. In a self-healing pipeline, classification errors are not merely reporting errors. They can lead to incorrect recovery actions. Therefore, confusion-matrix analysis is a practical diagnostic tool for improving the control loop.

B. Detection Alone Is Not Self-Healing

A central implication of this work is that detection alone should not be described as self-healing. Detection identifies an abnormal condition. Self-healing requires an action path after detection. That action path may involve retry, fallback,

quarantine, controlled continuation, escalation, or safe termination.

This distinction matters because many pipeline reliability systems generate alerts or validation failures but still leave operators to interpret the cause and choose a response. Such systems are useful, but they do not close the reliability loop. A self-healing system must connect observation to decision and decision to outcome.

The proposed framework addresses this gap by structuring the pipeline response around detection, classification, recovery, and telemetry. This structure makes it possible to evaluate whether the system is only noticing failures or actually managing them in a controlled way.

C. Recovery Must Be Governed

The evaluation also shows why recovery must be governed rather than automatic by default. Automated recovery can reduce manual effort, but unsafe automation can create downstream harm. For example, retry may be appropriate for a transient source issue, but it may be inappropriate for a schema violation that indicates a broken upstream contract. Similarly, fallback data may be acceptable for exploratory analytics but unacceptable for financial reporting, compliance reporting, or client-facing outputs.

For this reason, the framework treats containment and unresolved status as valid outcomes in some cases. This is a critical design point. A self-healing pipeline should not be judged only by the number of failures it automatically repairs. It should also be judged by whether it prevents unsafe propagation when repair is not justified.

This framing aligns the system with practical production needs. Engineering teams do not need automation that hides serious incidents. They need automation that reduces toil while preserving safety, auditability, and operator trust.

D. Relationship to Orchestration, Validation, and Observability

The framework should not be interpreted as a replacement for workflow orchestration, data-quality validation, or observability systems. Those systems remain necessary. Orchestration controls execution order and task scheduling. Validation checks whether data satisfies expected assumptions. Observability exposes operational signals and helps teams understand system state.

The contribution of the proposed framework is the integration layer above these capabilities. It uses signals from pipeline execution and validation to make reliability decisions. It does not merely schedule work, test data, or produce alerts. It connects those signals to classification, recovery, telemetry, and reproducible reporting.

This distinction is important for adoption. A production implementation would likely integrate with existing orchestration platforms, validation tools, logging systems, and governance controls. The framework provides a reliability-control pattern that can sit across those systems rather than requiring every organization to replace its existing stack.

E. Operational Implications for Data Engineering Teams

For data engineering teams, the main implication is that pipeline reliability should be designed as a lifecycle rather than as a set of disconnected checks. A pipeline incident typically requires several steps: noticing the issue, understanding its cause, deciding whether output can continue, taking action, verifying the result, and recording evidence. Manual handling of this lifecycle is slow and inconsistent.

The framework demonstrates how this lifecycle can be encoded in software. Even when full recovery is not possible, the system can still provide value by classifying the issue, containing invalid outputs, and producing an audit trail. This can reduce ambiguity during incident response and help teams prioritize human review.

The framework also encourages teams to define recovery policies explicitly. Without explicit policies, recovery behavior becomes ad hoc and difficult to audit. With explicit policies, teams can decide which failures are recoverable, which require quarantine, and which must stop the pipeline.

F. Implications for Production Adoption

The current implementation is a research framework, not a production deployment package. Production adoption would require additional engineering work. First, the framework would need to connect to real orchestration systems, storage platforms, validation tools, and monitoring systems. Second, recovery policies would need to be configured for specific business domains and risk levels. Third, the telemetry model would need to support operational audit requirements.

Production use would also require stronger security and governance controls. Automated remediation may need role-based approval, policy-as-code enforcement, change logging, and integration with incident-management systems. In regulated environments, recovery behavior may need to be reviewed and approved before it can affect production outputs.

The framework is therefore best interpreted as a reference architecture and experimental implementation. It demonstrates that the self-healing loop can be structured and evaluated, but production deployment would require organization-specific integration and validation.

G. Reproducibility as an Evaluation Requirement

A strength of the work is that the evaluation is packaged with reproducible artifacts. Reproducibility is important for self-healing systems because the claimed behavior must be inspectable. If a paper reports that a framework detects, classifies, and recovers from failures, reviewers should be able to inspect how those results were generated.

The use of synthetic data, controlled failure injection, telemetry outputs, and generated figures supports this goal. It makes the evaluation easier to reproduce than a study that depends on private production data. At the same time, reproducibility comes with a tradeoff: synthetic and controlled scenarios cannot capture the full diversity of production incidents.

This tradeoff is acceptable for the current stage of the work, but future evaluation should include production-like replay workloads, historical incident traces, and larger-scale execution environments.

H. Contribution Boundary

The contribution of this paper is not that detection, classification, recovery, or telemetry are individually new ideas. The contribution is the integration of these ideas into a controlled self-healing pipeline framework and the evaluation of the full loop under reproducible failure scenarios.

This boundary matters because overclaiming would weaken the paper. The framework should not be presented as a universal solution for all pipeline failures. It should be presented as a concrete implementation and evaluation of a reliability-control pattern for data pipelines.

The practical value is in showing how separate reliability mechanisms can be connected into an auditable and testable control loop. This creates a foundation for future work on production-scale self-healing systems.

I. Summary

The discussion highlights four main implications. First, self-healing requires more than detection. Second, recovery must be governed and safety-aware. Third, orchestration, validation, and observability are necessary but incomplete without a decision-and-action layer. Fourth, reproducible evaluation is essential for making self-healing claims credible.

The results support the feasibility of the proposed framework under controlled conditions. The limitations indicate that future work should extend the evaluation to larger workloads, real incident traces, distributed execution, streaming pipelines, and enterprise governance controls. Together, these findings position the framework as a practical step toward auditable self-healing data infrastructure.

VIII. LIMITATIONS AND THREATS TO VALIDITY

This section discusses the main limitations of the evaluation and the threats to validity that affect interpretation of the results. The framework is evaluated through controlled experiments using synthetic data, deterministic failure injection, and local execution artifacts. This design improves reproducibility, but it also limits how far the results can be generalized without additional production-scale validation.

A. Synthetic Data Limitation

The evaluation uses synthetic data rather than private enterprise datasets. This choice supports reproducibility because reviewers can regenerate the data and inspect the complete experiment workflow without access restrictions. It also avoids exposing sensitive operational data. However, synthetic data cannot fully reproduce the complexity of production data systems.

Production datasets often contain undocumented business rules, inconsistent historical conventions, hidden dependencies, delayed upstream feeds, manual corrections, and domain-specific exceptions. These characteristics can affect both failure detection and recovery behavior. A framework that performs well on synthetic data may still require additional feature engineering, policy configuration, and operational tuning before deployment in a real enterprise environment.

Therefore, the results should be interpreted as controlled evidence that the self-healing loop is implemented and testable, not as proof that the framework will perform identically on all real-world data pipelines.

B. Failure Scenario Coverage

The experiment design evaluates selected categories of pipeline failures, including source failures, schema failures, data-quality failures, runtime failures, healthy-control executions, and boundary-control scenarios. These categories represent common pipeline reliability problems, but they do not exhaust the full space of possible production failures.

Real pipelines may experience compound failures, partial upstream outages, intermittent dependency failures, permission changes, infrastructure degradation, silent data corruption, late-arriving data, backfill conflicts, downstream contract changes, or failures caused by external business processes. Some of these cases may produce signals that overlap across categories. Others may require recovery policies that are specific to an organization or regulatory environment.

The evaluated scenarios are sufficient to test the framework architecture under controlled conditions, but broader scenario coverage is needed before claiming comprehensive production readiness.

C. Controlled Failure Injection

Failure injection is intentionally controlled. The evaluation knows which scenarios are injected, while the detection and classification components must infer the failure from pipeline behavior. This setup enables repeatable measurement, but it may simplify the unpredictability of real incidents.

In production, failures are often noisy, incomplete, delayed, or masked by retries and partial outputs. A failure may also emerge gradually rather than appearing as a clean injected condition. Controlled injection therefore provides a useful experimental baseline, but it does not fully represent the uncertainty and ambiguity of live operational incidents.

Future work should evaluate the framework against historical incident logs, production-like replay workloads, and mixed-failure traces where the failure boundary is less clean.

D. Local Runtime Environment

Runtime results are measured in the local research environment used to execute the experiments. These measurements are useful for understanding the behavior of the implementation under the tested workload, but they should not be interpreted as universal performance benchmarks.

Runtime can vary based on machine resources, operating system behavior, Python version, dependency versions, disk

performance, logging configuration, and dataset size. A distributed execution environment may show different overhead patterns than a local execution environment. Similarly, streaming pipelines or large-scale batch jobs may require different recovery and telemetry designs.

The runtime results therefore provide local evidence of feasibility, not a final performance characterization for all deployment environments.

E. Recovery Generalizability

The recovery logic is evaluated under a bounded set of safe recovery actions. The framework can retry, contain, quarantine, fallback, or mark a failure as unresolved depending on the scenario. This design is intentionally conservative because unsafe automated recovery can cause more damage than the original failure.

However, production recovery behavior is highly context-dependent. A retry may be safe in one pipeline and unsafe in another. Quarantine may be appropriate for some datasets but operationally expensive for others. Fallback data may be acceptable for internal analytics but unacceptable for regulated reporting. The framework therefore requires policy configuration before production use.

The evaluation demonstrates that recovery decisions can be integrated into the self-healing loop, but it does not prove that a single recovery policy is appropriate for every organization or pipeline type.

F. Classification Validity

The classification component assigns detected failures to scenario categories. This supports recovery decisioning, but classification quality depends on the available features, telemetry signals, and scenario definitions. If two failure types produce similar symptoms, the classifier may require additional evidence to distinguish them reliably.

The confusion-matrix analysis helps identify classification behavior under the evaluated scenarios, but it does not eliminate the need for further validation. Real-world failures may involve overlapping causes. For example, a source failure can trigger downstream schema errors, and a schema error can create data-quality violations. In such cases, single-label classification may be insufficient.

Future versions of the framework may require hierarchical, multi-label, or confidence-aware classification to better represent complex incidents.

G. Policy and Governance Boundaries

The framework treats recovery as a governed action rather than an automatic response to every detected failure. This is necessary because self-healing systems can introduce operational risk if they modify pipeline behavior without appropriate constraints.

The current evaluation demonstrates policy-aware framing, but it does not implement a full enterprise governance model with role-based approval workflows, regulatory controls, audit-retention policies, or organization-specific risk thresholds. In

regulated industries, these controls would be necessary before automated remediation could be trusted in production.

The results should therefore be interpreted as evidence for the technical control loop, not as a complete compliance framework.

H. External Validity

External validity concerns how well the results generalize beyond the tested environment. The evaluated framework is designed around a reproducible research implementation. Production systems may use different orchestrators, storage systems, data contracts, quality tools, observability platforms, and deployment patterns.

The architectural concepts are intended to be portable: detect, classify, decide, recover, and audit. However, the implementation details may need adaptation for specific platforms such as distributed processing engines, cloud-native orchestration systems, event-driven pipelines, or streaming data architectures.

The framework should therefore be viewed as a reference architecture and experimental implementation rather than a plug-and-play replacement for all production reliability tooling.

I. Internal Validity

Internal validity concerns whether the observed results are caused by the framework behavior rather than by flaws in the experiment. The evaluation reduces internal validity risks by using controlled scenarios, repeatable scripts, preserved artifacts, and compile-tested reporting outputs. The repository also includes tests that support implementation-level confidence.

However, internal validity risks remain. Bugs in the data generator, failure injection logic, detection rules, classification logic, recovery logic, or aggregation scripts could affect reported outcomes. The evaluation artifacts reduce this risk by making the workflow inspectable, but they do not eliminate it completely.

A stronger future evaluation would include independent replication, additional unit and integration tests, mutation-style failure tests, and cross-environment execution.

J. Construct Validity

Construct validity concerns whether the metrics measure the intended concepts. Detection accuracy, classification behavior, recovery status, runtime distribution, and confusion-matrix analysis are useful indicators of self-healing behavior, but they do not capture every operational concern.

For example, the current evaluation does not directly measure operator workload reduction, incident-resolution time in a live team, business impact avoided, downstream trust improvement, or compliance-review efficiency. These are important operational outcomes, but they require production or field-study evaluation.

The current metrics therefore measure technical feasibility and controlled reliability behavior. They should not be interpreted as complete evidence of organizational impact.

K. Reproducibility Boundaries

The project provides code, generated artifacts, figures, and manuscript assets to support reproducibility. This improves transparency, but reproducibility still depends on environment setup, dependency installation, operating system behavior, and correct execution of the scripts.

The artifacts support verification of the reported workflow, but they do not guarantee identical runtime measurements across machines. Minor differences in dependency versions or local execution conditions may affect timing results while leaving detection and classification behavior unchanged.

For this reason, runtime should be interpreted as environment-specific, while scenario outcomes and artifact structure provide the stronger reproducibility signal.

L. Summary

The main limitation of the study is that it evaluates a self-healing pipeline framework under controlled, reproducible conditions rather than through large-scale production deployment. This is an appropriate first step for demonstrating architecture, implementation, and experimental feasibility, but it is not sufficient for universal claims.

The results show that the framework can integrate detection, classification, recovery decisioning, telemetry, and reproducible reporting under the evaluated scenarios. Future work should extend the evaluation to production-like datasets, larger workloads, historical incident traces, multi-label failure diagnosis, distributed execution, streaming pipelines, and enterprise governance controls.

IX. CONCLUSION AND FUTURE WORK

This paper presented a reliability-aware, policy-constrained self-healing control plane for data pipelines. The framework models self-healing as a staged reliability lifecycle rather than as a single detection or retry mechanism. The lifecycle includes failure detection, root-cause classification, remediation selection, policy approval, execution, post-recovery validation, rollback or escalation, and evidence preservation.

The evaluation used a reproducible synthetic failure-injection setup with 780 total trials, including injected failures, healthy controls, and boundary controls. Under the evaluated controlled experiment configuration, the framework achieved 100% failure detection accuracy and 100% root-cause classification accuracy. Verified recovery was achieved in 52.17% of injected failure trials. Runtime was measured across 780 records, with a mean self-healing cycle runtime of 12.609 ms, a median of 10.609 ms, and p95 of 26.666 ms.

These results support the feasibility of the proposed framework under controlled experimental conditions. More importantly, they show why detection, diagnosis, and verified recovery should be evaluated separately. Accurate detection does not guarantee safe remediation. Correct classification does not guarantee that a pipeline should continue. A recovery action should be considered successful only when it is policy-approved, executed, and validated after execution.

The main conclusion is that self-healing data pipelines should be treated as governed reliability-control systems. A useful framework must not only identify failures, but also determine whether automated remediation is safe, whether invalid output propagation should be blocked, and whether sufficient evidence has been preserved for audit and later inspection. This distinction is especially important in enterprise and regulated data environments, where unrestricted automation can create operational or compliance risk.

The study also has clear boundaries. The evaluation uses synthetic data, controlled failure injection, and a local research execution environment. These choices improve reproducibility but limit direct generalization to all production pipelines. The results should therefore be interpreted as evidence of architectural and experimental feasibility rather than proof of universal production readiness.

Future work should extend the framework in several directions. First, the evaluation should be repeated using production-like datasets, historical incident traces, and larger workloads. Second, the failure taxonomy should be expanded to include compound failures, intermittent upstream outages, delayed data, permission changes, and downstream contract violations. Third, the framework should be compared against alert-only, retry-only, and static-rule baselines under identical scenarios. Fourth, future versions should explore probabilistic, confidence-aware, hierarchical, or multi-label root-cause diagnosis. Finally, production-oriented work should study integration with orchestration platforms, observability systems, policy-as-code engines, incident-management workflows, and enterprise governance controls.

Overall, the paper contributes a reproducible implementation and evaluation of an end-to-end self-healing control loop for data pipelines. The work demonstrates that self-healing claims are strongest when they are supported by separate evidence for detection, classification, recovery, runtime behavior, and auditability.

DATA AND ARTIFACT AVAILABILITY

The evaluation uses generated synthetic data and controlled failure-injection scenarios. No proprietary enterprise dataset, private customer record, or human-subject dataset is required to reproduce the reported experiments. The working manuscript package includes the LaTeX source, bibliography, section files, tables, and publication-ready result figures used in this manuscript. The broader project repository contains the implementation, synthetic-data generation workflow, failure-injection logic, evaluation outputs, derived summaries, confusion-matrix data, and figure-generation scripts used to reproduce the reported results.

At the working-manuscript stage, this paper does not claim a final public DOI or archival repository URL. Before journal submission, the artifact package should be deposited in a stable public repository or archival service, and this statement should be updated with the final URL or DOI.

REFERENCES

- [1] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, “Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers,” 2023.
- [2] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems*, 2015.
- [3] D. Xin, H. Miao, A. Parameswaran, and N. Polyzotis, “Production machine learning pipelines: Empirical analysis and optimization opportunities,” 2021.
- [4] Apache Software Foundation, “Dags — airflow documentation,” <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>, 2026, accessed: 2026-06-15.
- [5] Great Expectations, “Try gx core,” https://docs.greatexpectations.io/docs/core/introduction/try_gx/, 2026, accessed: 2026-06-15.
- [6] dbt Labs, “Add data tests to your dag,” <https://docs.getdbt.com/docs/build/data-tests>, 2026, accessed: 2026-06-15.
- [7] Open Policy Agent, “Open policy agent documentation,” <https://www.openpolicyagent.org/docs>, 2026, accessed: 2026-06-15.
- [8] Google, “Monitoring distributed systems,” <https://sre.google/sre-book/monitoring-distributed-systems/>, 2026, accessed: 2026-06-15.
- [9] D. Tu, Y. He, W. Cui, S. Ge, H. Zhang, H. Shi, D. Zhang, and S. Chaudhuri, “Auto-validate by-history: Auto-program data quality constraints to validate recurring data pipelines,” 2023.
- [10] T. Rehberger, T. Hütter, L. Ehrlinger, and W. Wöß, “Evaluating data quality tools: Measurement capabilities and llm integration,” 2026.
- [11] J. Yasmin, J. Wang, Y. Tian, and B. Adams, “An empirical study of developers’ challenges in implementing workflows as code: A case study on apache airflow,” 2024.
- [12] P. Notaro, J. Cardoso, and M. Gerndt, “A systematic mapping study in aiops,” 2020.
- [13] A. Saha and S. C. H. Hoi, “Mining root cause knowledge from cloud service incident investigations for aiops,” 2022.
- [14] J. O. Kephart and D. M. Chess, “The vision of autonomic computing,” *Computer*, vol. 36, no. 1, pp. 41–50, 2003.